

Calcolo Numerico = elementi di base di Analisi Numerica (matematica applicata)

Di cosa si occupa l'Analisi Numerica (in breve)

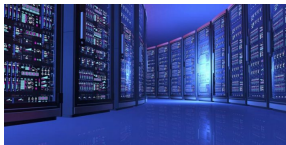
Lo scopo dell'Analisi Numerica è risolvere problemi matematici mediante un calcolatore.



Ogni calcolatore:

- è una macchina in grado di eseguire velocemente una grande quantità di operazioni aritmetiche;
- non è in grado di elaborare entità astratte ma solo dati rappresentabili in forma di numeri;
- ha bisogno di una lista precisa e ordinata di istruzioni da eseguire per produrre il risultato richiesto.

L'uso del calcolatore impone una formulazione opportuna dei problemi da risolvere e la progettazione accurata della procedura con cui calcolarle.



Problemi matematici analitici e numerici: un esempio semplice

Problema matematico: data la funzione $f(x) = \sin(x^2)$, calcolarne la derivata.

Soluzione analitica (astratta) - calculus

Applicando i teoremi e le definizioni dell'analisi matematica, si ricava l'espressione analitica della funzione derivata:

$$f'(x) = 2x \cos(x^2)$$



Il dato del problema analitico è una funzione, $f(x)$, il risultato è una funzione, $f'(x)$.

Soluzione numerica (calcolabile) - computation

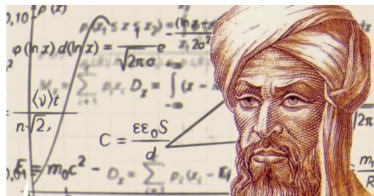
Applicando i teoremi e i principi dell'analisi numerica, si definisce una procedura che, dato un numero $x \in \mathbb{R}$, calcola un numero y che approssima bene la derivata prima di f in x

$$y \simeq f'(x)$$

Il dato del problema *numerico* è un numero reale, x , il risultato è un numero reale, y .

- Per ottenere una soluzione numerica, occorre fornire al calcolatore i dati e la lista precisa delle operazioni da eseguire a partire da essi;
- La lista deve contenere solo operazioni che il calcolatore è in grado di interpretare ed eseguire;
- in altre parole è necessario progettare un algoritmo

Etimologia: dal nome d'origine, al-Khuwarizmi, del matematico arabo Muhammad ibn Musa del IX sec. (così chiamato perché nativo di Khwarizm, regione dell'Asia Centrale) che introdusse in Europa le cifre arabe e il sistema decimale – Termine che indicò nel medioevo i procedimenti di calcolo basati sull'uso delle cifre arabiche.



Definizione generale

Un algoritmo è una procedura che, a partire da un insieme di dati in ingresso, produce un insieme di risultati in uscita mediante un numero finito di passi definiti in modo univoco.

DATI

PROCEDIMENTO

RISULTATI

INGREDIENTI

- Spaghetti 320 g
- Tuorli di uova medie 6
- Pepe nero q.b.
- Guanciale 150 g
- Pecorino romano 50 g



Per preparare gli spaghetti alla carbonara cominciate mettendo sul fuoco una pentola con l'acqua salata per cuocere la pasta. Nel frattempo eliminate la cotenna dal guanciale **1** e tagliatelo prima a fette e poi a striccioline spesse circa 1cm **2**. La cotenna avanzata potrà essere riutilizzata per insaporire altre preparazioni. Versate i pezzetti in una padella antiaderente **3** e rosolate per circa 15 minuti a fiamma media, fate attenzione a non bruciarli altrimenti rilascerà un aroma troppo forte.



Nel frattempo tuffate gli spaghetti nell'acqua bollente **4** e cuoceteli per il tempo indicato sulla confezione. Intanto versate i tuorli in una ciotola **5**, aggiungete anche la maggior parte del Pecorino previsto dalla ricetta **6**; la parte restante servirà per guarnire la pasta.



Insaporite con il pepe nero **7**, amalgamate tutto con una frusta a mano **8**. Aggiungete un cucchiaino di acqua di cottura per diluire il composto e mescolate **9**.



Insaporite con il pepe nero **7**, amalgamate tutto con una frusta a mano **8**. Aggiungete un cucchiaino di acqua di cottura per diluire il composto e mescolate **9**.



Intanto il guanciale sarà giunto a cottura, spegnete il fuoco e tenetelo da parte **10**. Scolate la pasta al dente direttamente nel tegame con il guanciale **11** e saltatela brevemente per insaporirla. Togliete dal fuoco e versate il composto di uova e Pecorino **12** nel tegame. Mescolate velocemente per amalgamare.



Per renderla ben cremosa, al bisogno, potete aggiungere poca acqua di cottura della pasta **13**. Servite subito gli spaghetti alla carbonara insaporendoli con il Pecorino avanzato **14** e il pepe nero macinato a piacere **15**.



Definizione generale

Un algoritmo è una procedura che, a partire da un insieme di dati in ingresso, produce un insieme di risultati in uscita mediante un numero finito di passi definiti in modo univoco.

Negli algoritmi *numerici*,

- i dati in input e le soluzioni in output sono **numeri reali**.
- i passi sono costituiti da **successioni delle quattro operazioni aritmetiche fondamentali**, per poter essere eseguiti da un **calcolatore**.

L'Analisi Numerica comprende i seguenti aspetti:

- l'analisi dei problemi provenienti dal mondo della tecnologia e delle scienze applicate
- la modellazione di tali problemi in forma di problemi matematici
- l'analisi di questi problemi matematici, in forma analitica e numerica (es. esistenza ed unicità delle soluzioni, proprietà di regolarità,...)
- l'analisi e lo sviluppo degli algoritmi numerici
- gli aspetti implementativi degli algoritmi

- Prenderemo in esame alcuni problemi matematici semplici ma fondamentali;
- Studieremo gli algoritmi numerici opportuni per risolverli, analizzando le loro proprietà teoriche e confrontandoli tra loro;
- Realizzeremo un'implementazione degli algoritmi in ambiente Python (NumPy, SciPy).

Premessa

Un fatto fondamentale

- Lo strumento che eseguirà i passi degli algoritmi numerici è il calcolatore
- I dati e i risultati, sia quelli finali che quelli intermedi, di ogni algoritmo numerico devono essere memorizzati (= rappresentati) nei dispositivi di memoria del calcolatore.
- Il calcolatore ha uno spazio di memoria finito

Questo fatto ha un'importanza enorme nella progettazione e nell'analisi dei problemi e degli algoritmi numerici.

Punto di partenza del corso

- ① In che modo il calcolatore memorizza i numeri interi e reali?
- ② In che modo il calcolatore esegue le operazioni aritmetiche sui numeri interi e reali?

Consideriamo il seguente pseudocodice e le corrispondenti istruzioni Matlab

```
 $u \leftarrow 1$   
 $i \leftarrow 0$   
while  $u + 1 > 1$   
   $u \leftarrow u/2$   
   $i \leftarrow i + 1$   
  
stampa  $i$ 
```

```
u = 1.  
i = 0  
while u + 1 > 1:  
    u = u/2  
    i +=1  
  
print(u,i)
```

Risultato:

L'output del programma è la stampa

1.1102230246251565e-16 53

Significa che per il calcolatore è vera l'uguaglianza

$$1 + \frac{1}{2^{53}} = 1$$



Nella prima parte del corso vedremo come questo sia una diretta conseguenza di come il calcolatore *rappresenta* i numeri reali e come esegue le operazioni aritmetiche.

La rappresentazione dei numeri

Sommario della lezione:

- Richiami sulla notazione posizionale
- Rappresentazione di interi e reali in basi non decimali
 - Algoritmo delle divisioni successive
 - Algoritmo delle moltiplicazioni successive

distinguiamo il concetto di quantità espressa da un numero e la sua rappresentazione.

NUMERO

↙
ENTITA' ASTRATTA
quantità "sette"
univocamente determinata

↘
RAPPRESENTAZIONE
(7, VII, (111)₂, ...)
molteplice a seconda dei
criteri di rappresentazione adottati.

La convenzione da noi usata per esprimere un numero è detta *notazione posizionale*, in cui ogni cifra assume un valore diverso a seconda della sua posizione all'interno della stringa usata per rappresentarlo.

Cifre	7	5	9	=	$7 \cdot 10^2$	+	$5 \cdot 10^1$	+	$9 \cdot 10^0$
Posizione	2	1	0						
Base	10								

- La notazione posizionale decimale è caratterizzata da una *base* $\beta = 10$, dalle posizioni delle cifre che costituiscono gli *esponenti* della base e da un insieme di *simboli*, le cifre $0, 1, 2, 3, \dots$, ciascuna delle quali indica un valore compreso tra 0 e 9.
- Mantenendo la convenzione posizionale, possiamo estendere il discorso per qualsiasi base $\beta > 1$, introducendo un insieme di simboli

BASE	SIMBOLI
2	$\mathcal{S} = \{0, 1\}$
8	$\mathcal{S} = \{0, 1, 2, 3, 4, 5, 6, 7\}$
16	$\mathcal{S} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$

- L'insieme dei simboli contiene le cifre corrispondenti ai valori $0, 1, \dots, \beta - 1$.

- Data una base β e il corrispondente insieme di simboli $\mathcal{S}$, ogni numero naturale N si può rappresentare in modo univoco con una stringa di cifre

$$(c_p c_{p-1} \dots c_1 c_0)_\beta$$

dove $p \in \mathbb{N}$.

- Se, con un piccolo abuso di notazione, indichiamo con c_i anche il valore della cifra e identifichiamo il numero con la sua rappresentazione (univoca), allora il valore del numero è

$$N = (c_p c_{p-1} \dots c_1 c_0)_\beta = c_p \beta^p + c_{p-1} \beta^{p-1} + \dots + c_1 \beta + c_0$$

Cifre significative della rappresentazione

$$N = (c_p c_{p-1} \cdots c_1 c_0)_\beta = c_p \beta^p + c_{p-1} \beta^{p-1} + \dots + c_1 \beta + c_0$$

Supponiamo $c_p \neq 0$.

Dato $t \leq p$,

- le t cifre **meno significative** della rappresentazione in base β del numero $N \in \mathbb{N}$ sono i pesi delle potenze più piccole della base:

$$(c_p c_{p-1} \cdots c_t \underbrace{c_{t-1} c_1 c_0})_\beta$$

Si contano da destra;

- le t cifre **più significative** nella rappresentazione in base β del numero $N \in \mathbb{N}$ sono i pesi delle potenze più grandi della base:

$$(\underbrace{c_p c_{p-1} \cdots c_{p-t+1}} \cdots c_1 c_0)_\beta$$

Si contano da sinistra partendo dalla prima cifra non nulla della rappresentazione.

"trecentosettantadue" =

$$(372)_{10} = 3 \cdot 10^2 + 7 \cdot 10^1 + 2 \cdot 10^0$$

$$(174)_{16} = 1 \cdot 16^2 + 7 \cdot 16^1 + 4 \cdot 16^0$$

$$(564)_8 = 5 \cdot 8^3 + 6 \cdot 8^1 + 4 \cdot 8^0$$

$$(101110100)_2 = 1 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 1 \cdot 2^5 + \\ + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$$

"duecentoottantasette" =

$$(287)_{10} = (100011111)_2 = (10133)_4$$

$$(437)_8 = (11F)_{16} = (8V)_{32}.$$

Osservazione

Al'augmentare della base, aumenta il numero dei simboli. Al contrario, con una base più grande sono in genere necessarie meno cifre per rappresentare uno stesso valore.

La rappresentazione posizionale di numeri reali < 1 in base β

Un numero $\alpha \in \mathbb{R}$, $0 \leq \alpha < 1$, si rappresenta in una base fissata $\beta > 1$ come

$$\alpha = (0.a_1a_2...a_t a_{t+1}...)_{\beta} = a_1\beta^{-1} + a_2\beta^{-2} + ... + a_t\beta^{-t} + a_{t+1}\beta^{-t-1} + ...$$

dove “.” è il *punto radice* e a_i sono cifre appartenenti all'insieme dei simboli associato alla base β .

Lo zero a sinistra del punto radice può essere omissso.

Tutto ciò che è a destra del punto radice è il peso di una potenza negativa della base.

Esempi:

β	rappresentazione	valore
10	$(0.1)_{10}$	$1/10$
10	$(0.11)_{10}$	$1/10 + 1/100$
2	$(0.1)_2$	$1/2$
2	$(0.11)_2$	$1/2 + 1/4$
16	$(0.1)_{16}$	16^{-1}

La somma delle potenze negative si può scrivere anche come

$$\alpha = (0.a_1a_2...a_t a_{t+1}...)_{\beta} = \sum_{i=1}^{\infty} a_i\beta^{-i}.$$

Si osservi che per rappresentare certi valori (numeri periodici, irrazionali,...) è necessario un numero infinito di cifre.

Rappresentazione di un qualsiasi numero reale

Un numero reale α si può ottenere come somma della sua parte intera più la sua parte frazionaria. Quindi, mettendo insieme i due contributi e tenendo conto del segno si ha

$$\begin{aligned}\alpha &= \text{segno}(\alpha) a_1 a_2 \dots a_p \cdot a_{p+1} a_{p+2} a_{p+3} \dots \\ &= \text{segno}(\alpha) (a_1 \beta^{p-1} + a_2 \beta^{p-2} + \dots + a_p + a_{p+1} \beta^{-1} + a_{p+2} \beta^{-2} + \dots)\end{aligned}$$

Raccogliendo un fattore β^p tra tutti i termini, si può scrivere

$$\begin{aligned}\alpha &= \text{segno}(\alpha) (a_1 \beta^{-1} + a_2 \beta^{-2} + a_3 \beta^{-3} + \dots) \beta^p \\ &= \text{segno}(\alpha) \sum_{i=1}^{\infty} (a_i \beta^{-i}) \beta^p \\ &= \text{segno}(\alpha) m \beta^p\end{aligned}$$

dove

- segno vale +1 o -1 a seconda del **segno** di α ;
- $m = \sum_{i=1}^{\infty} (a_i \beta^{-i}) = (0.a_1 a_2 \dots)_{\beta}$ è un numero reale < 1 chiamato **mantissa**;
- β^p è un intero chiamato **parte esponente** di α .

Rappresentazione di numeri reali

Forma scientifica normalizzata

Le rappresentazioni del tipo segno–mantissa–esponente

$$\alpha = \text{segno}(\alpha)(0.a_1a_2\dots)\beta^p$$

si dicono in *forma scientifica*; se $a_1 \neq 0$ si parla di forma scientifica *normalizzata*.

ESEMPI

$(372)_{10}$	mista
$.372 \cdot 10^3$	normalizzata
$.0372 \cdot 10^4$	scientifica
$(3.141592\dots)_{10}$	mista
$.3141592\dots \cdot 10^1$	normalizzata
$.3243F\dots \cdot 16^1$	normalizzata
$(3.243F\dots)_{16}$	mista

L'unicità della rappresentazione si ha nel caso della forma scientifica normalizzata.

Algoritmo delle divisioni successive

Serve per rappresentare un intero positivo $\alpha \in \mathbb{N}$ da base 10 ad una diversa base β .

L'algoritmo consiste nell'eseguire la divisione intera (quoziente e resto) del numero α per β finchè non si giunge ad un quoziente nullo.

Esempio

Rappresentazione del numero $(1972)_{10}$ in base 2.

base 2

$1972 : 2 = 986$	resto 0
$986 : 2 = 493$	resto 0
$493 : 2 = 246$	resto 1
$246 : 2 = 123$	resto 0
$123 : 2 = 61$	resto 1
$61 : 2 = 30$	resto 1
$30 : 2 = 15$	resto 0
$15 : 2 = 7$	resto 1
$7 : 2 = 3$	resto 1
$3 : 2 = 1$	resto 1
$1 : 2 = 0$	resto 1

$$(1972)_{10} = (11110110100)_2$$

Algoritmo delle moltiplicazioni successive

Serve per rappresentare un reale decimale $\alpha \in [0, 1)$ in una base $\beta > 1$. Si tratta di determinare le cifre della rappresentazione

$$\alpha = (.a_1a_2a_3...)_{\beta} = a_1\beta^{-1} + a_2\beta^{-2} + a_3\beta^{-3} + \dots$$

moltiplicando il numero α per la base β .

Esempio

$(0.1)_{10}$

base 2

$$0.1 \times 2 = 0.2 \quad \text{p. intera } 0$$

$$0.2 \times 2 = 0.4 \quad \text{p. intera } 0$$

$$0.4 \times 2 = 0.8 \quad \text{p. intera } 0$$

$$0.8 \times 2 = 1.6 \quad \text{p. intera } 1$$

$$0.6 \times 2 = 1.2 \quad \text{p. intera } 1$$

$$0.2 \times 2 = 0.4 \quad \text{p. intera } 0$$

...

$$(0.1)_{10} = (0.000\overline{1100})_2$$

Conversione di un reale da base 10 a base β

Ricordando che ogni numero si esprime come somma della sua parte intera e della sua parte frazionaria, gli algoritmi delle divisioni e delle moltiplicazioni successive possono essere impiegati per calcolare la conversione di base di un qualunque reale.

- 1 Determinare $|\alpha|$, ricordando il segno.
- 2 Determinare la parte intera $\lfloor |\alpha| \rfloor$ ed eseguire la conversione con l'algoritmo delle divisioni successive.
- 3 Determinare la parte frazionaria $|\alpha| - \lfloor |\alpha| \rfloor$ ed eseguire la conversione con l'algoritmo delle moltiplicazioni successive.
- 4 Scrivere il segno, la conversione della parte intera, il punto radice, la conversione della parte frazionaria.

Esempio

$\alpha = -(25.375)_{10}$. Convertire in base 2.

- 1 $|\alpha| = 25.375$; segno='-'.
- 2 $\lfloor |\alpha| \rfloor = 25$; $(25)_{10} = (11001)_2$.
- 3 $|\alpha| - \lfloor |\alpha| \rfloor = .375$; $(.375)_{10} = (.011)_2$.
- 4 $\alpha = -(11001.011)_2$.

I numeri di macchina - Numeri Fixed Point

Sommario della lezione:

- Rappresentazione degli interi in formato fixed point
- Operazioni con i numeri fixed point

Rappresentazione fixed point degli interi

Base 2, $t + 1$ bit

Al momento della assegnazione di un valore intero, questo valore viene trasformato in una stringa di cifre binarie secondo le regole del formato *fixed point* (virgola fissa).

Il formato fixed point dipende da un parametro che in diciamo con t

- Viene riservato in memoria uno spazio di $t + 1$ bit (formati standard $t + 1 = 16$, $t + 1 = 32$);
- Indichiamo con $fi(N)$ la stringa di cifre binarie ottenuta applicando le regole del formato all'intero N .

$$fi(N) = \begin{array}{|c|c|c|c|c|c|c|} \hline c_t & c_{t-1} & \dots & \dots & \dots & c_1 & c_0 \\ \hline \end{array}$$

Regole della rappresentazione fixed point degli interi

- se $N \geq 0$, si memorizzano le $t + 1$ cifre meno significative, $d_t, d_{t-1}, \dots, d_0$, della sua rappresentazione binaria (eventuali zeri a sinistra compresi)

$$fi(N) = \begin{array}{|c|c|c|c|c|c|c|} \hline d_t & d_{t-1} & \cdots & \cdots & \cdots & d_1 & d_0 \\ \hline \end{array}$$

- se $N < 0$, si prendono le $t + 1$ cifre meno significative, $d_t, d_{t-1}, \dots, d_0$, della rappresentazione binaria di $|N|$ (eventuali zeri a sinistra compresi) e si calcola la rappresentazione in *complemento a 2* nel seguente modo:

$$\tilde{d}_i = \begin{cases} 1 & \text{se } d_i = 0 \\ 0 & \text{se } d_i = 1 \end{cases} \Rightarrow \begin{array}{ccccccc} \tilde{d}_t & \tilde{d}_{t-1} & \cdots & \tilde{d}_0 & + & & \\ & & & & 1 & = & \\ \hline c_t & c_{t-1} & \cdots & c_0 & & & \end{array}$$

(dove la somma è in aritmetica binaria)

$$fi(N) = \begin{array}{|c|c|c|c|c|c|c|} \hline c_t & c_{t-1} & \cdots & \cdots & \cdots & c_1 & c_0 \\ \hline \end{array}$$

Esempi

$$t + 1 = 16$$

- $N = 1235 = (10011010011)_2$

$$fi(N) = \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|}\hline 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 1 \\ \hline\end{array}$$

- $N = -1235 = -(10011010011)_2$

- ① Rappresentazione binaria su 16 bit: $|N| = 0000010011010011$
- ② Scambio dei bit: 1111101100101100
- ③ Somma dell'unità: 1111101100101101

$$fi(N) = \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|}\hline 1 & 1 & 1 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ \hline\end{array}$$

Insieme dei numeri fixed point in base 2 con $t + 1 = 4$

$fi(N)$	N
1111	-1
1110	-2
1101	-3
1100	-4
1011	-5
1010	-6
1001	-7
1000	-8
0111	7
0110	6
0101	5
0100	4
0011	3
0010	2
0001	1
0000	0

- I numeri interi nell'intervallo $[-8, 7]$ esauriscono tutte le possibili combinazioni di bit ottenibili su 4 cifre
- Per un generico t l'intervallo corrispondente è $[-2^t, 2^t - 1]$
- Per i formati standard si ha

tipo	$t + 1$	-2^t	$2^t - 1$
short (int)	16	-32768	32767
long int	32	$-2.147483648 \cdot 10^9$	$2.147483647 \cdot 10^9$

- E gli altri interi?

Esempi

base 2 con $t + 1 = 4$

Regola:

Si prendono sempre le $t + 1$ cifre meno significative della rappresentazione binaria del numero, se è positivo, del suo complemento a 2 su $t + 1$ cifre, se è negativo.

- $N = -9$, $N = -(1001)_2$, rappresentazione in complemento a 2 su 4 cifre
 - 1001
 - 0110
 - 0111 $\Rightarrow fi(N) = 0111$
- $N = 23$, $N = (10111)_2$ $\Rightarrow fi(N) = 0111$
- -25 , $N = -(11001)_2$, rappresentazione in complemento a 2 su 4 cifre
 - 1001
 - 0110
 - 0111 $\Rightarrow fi(N) = 0111$
- ...

Insieme dei numeri fixed point in base 2 con $t + 1 = 4$

	$fi(N)$	N
	1111	-1
	1110	-2
	1101	-3
	1100	-4
	1011	-5
	1010	-6
	1001	-7
-9 ↘	1000	-8
-25 →	0111	7
23 ↗	0110	6
	0101	5
	0100	4
	0011	3
	0010	2
	0001	1
	0000	0

- Per questo calcolatore vale l'uguaglianza $-9 = -25 = 23 = 7$ 😲
- Il caso in cui la perdita di informazione si ha nella rappresentazione di un intero troppo piccolo, ossia $N < -2^t$, è chiamato **underflow intero** (es. -9,-25)
- Il caso in cui la perdita di informazione si ha nella rappresentazione di un intero troppo grande, ossia $N > 2^t - 1$, è chiamato **overflow intero** (es. -23)
- L'intervallo $[-2^t, 2^t - 1]$ è chiamato **intervallo di esatta rappresentazione degli interi**

Aritmetica dei numeri fixed point in base 2 con $t + 1 = 4$

	$fi(N)$	N
	1111	-1
	1110	-2
	1101	-3
	1100	-4
	1011	-5
	1010	-6
	1001	-7
-9 ↘	1000	-8
-25 →	0111	7
23 ↗	0110	6
	0101	5
	0100	4
	0011	3
	0010	2
	0001	1
	0000	0

- Operando con numeri compresi nell'intervallo $[-2^t, 2^t - 1]$ si può ottenere un risultato che non appartiene all'intervallo
- Esempio: $7 + 4 = 11 = (1001)_2 = fi(-7)$
 $\Rightarrow$ per questo calcolatore $7 + 4 = -7$ 😱
- Esempio: $7 \cdot 3 = 21 = (10101)_2$ in questo caso vengono memorizzate le $t + 1$ cifre meno significative della rappresentazione binaria del risultato: $\Rightarrow (0101)_2 \Rightarrow$
 $7 \cdot 3 = 5$ 😱

- Il calcolatore rappresenta gli interi in formato fixed point.
- Solo un sottoinsieme limitato di interi è rappresentato esattamente al calcolatore.
- Numeri troppo grandi o troppo piccoli (overflow o underflow) vengono rappresentati con una perdita di informazione, ossia con un *errore*.
- I risultati delle operazioni aritmetiche tra numeri fixed point vengono rappresentati all'interno dell'intervallo $[-2^t, 2^t - 1]$ tramite le $t + 1$ cifre meno significative.
- In caso di underflow o di overflow, anche il risultato di un'operazione viene rappresentato con un errore